

The ScrapSnatcher class extends the existing abstract Creature class and implements both Spawnable and Infectable. This allows it to participate in the existing creature turn-

taking structure while also supporting spawn effects and parasite infection through interfaces. When the Scrap Snatcher is in normal state, it uses `SnatchDepositResourceBehaviour` to move randomly. If the randomly selected destination contains an item with the Depositable capability, the behaviour creates a `MoveSnatchResourceAction`, allowing the Snatcher to move and immediately snatch the resource. This avoids checking concrete item classes such as `AluminiumScrap`, `IndustrialFan`, or `AlienArtifact`, which reduces coupling and avoids the previous instanceof design smell.

The loot explosion is handled by a separate `LootExplosion` utility class. A Scrap Snatcher triggers this effect through its `spawnEffect()` method when spawned by a Hole, Vent, or deprecated fleshy tree. This design keeps spawning sources simple: they only create and place spawnable actors, while the spawned actor decides its own special spawn effect. This follows the Single Responsibility Principle because Hole and Vent do not need to understand Scrap Snatcher-specific item-dropping rules.

The infection logic is handled using the existing `Infectable` abstraction. Parasite uses `InfectBehaviour` and `InfectAction` to infect valid targets. When a Scrap Snatcher is infected, it gains `InfectedSnatcherStatus`. The infected status applies one damage per turn and prints feedback to the console using the engine `Display` class. In its `playTurn()` method, `ScrapSnatcher` checks whether it has the infected status: if not infected, it snatches resources; if infected, it stops hoarding and uses `AttackWorkerBehaviour` to attack adjacent workers. This keeps the state-based behaviour change inside the Scrap Snatcher rather than spreading condition checks across unrelated classes.

For the 99-Deprecated map, I added `DeprecatedFleshySprout`, `DeprecatedFleshyMatureTree`, and `FleshyMonolith`. These classes represent the special tree lifecycle required by REQ2 without modifying the existing 20-Overflow fleshy tree behaviour. `DeprecatedFleshySprout` spawns an Undead and can grow directly into `DeprecatedFleshyMatureTree`, skipping the sapling stage. `DeprecatedFleshyMatureTree` spawns a `ScrapSnatcher` and can grow into `FleshyMonolith`. `FleshyMonolith` extends `Tree` directly rather than `FleshyTree` because it no longer behaves like a spawning tree; it acts as an involuntary teleporter. This avoids forcing the monolith into an unsuitable spawning contract and helps preserve the Liskov Substitution Principle.

To reduce duplicated code, shared fleshy tree proximity spawning logic is kept in `FleshyTree`. Subclasses only decide what actor to spawn or what growth transition to perform. This improves cohesion because each class focuses on its own stage-specific behaviour, while shared logic remains centralised.

Design Option	Pros	Cons	Decision
Put all Scrap Snatcher logic directly inside ScrapSnatcher	Simple and fewer classes	Low cohesion; actor class becomes responsible for movement, item detection, infection damage, and loot explosion	Rejected
Use separate behaviour, action, status, and utility classes	Higher cohesion, easier to test and extend, follows SRP	More classes are required	Chosen
Check concrete item classes when snatching resources	Easy to implement quickly	High coupling and violates OCP when new depositable resources are added	Rejected
Use the Depositable abstraction	Supports future depositable items without changing Snatcher logic	Requires items to expose the correct capability	Chosen
Modify existing FleshySprout and FleshyMatureTree for 99-Deprecated	Fewer classes	Could break 20-Overflow behaviour and mix map-specific rules	Rejected
Add map-specific deprecated fleshy tree subclasses	Preserves existing behaviour and follows OCP	Adds more classes	Chosen

Overall, this design improves on previous feedback by avoiding concrete type checks, keeping action-target relationships explicit, using interface-based capabilities, and ensuring continuous effects such as infection damage provide visible feedback. It also keeps the game engine unchanged and integrates with existing engine concepts such as Actor, Action, Ground, Status, and Location.